

Tech Spec: Athena Phase 2 for ROI

Purpose of this Document

Relevant links

Section Owner: @Jonathan Falkenstein and @Alagu Manikandan Esakkimuthu

System Context

Design Goals

In the short-term...

In the medium-to-long-term...

Proposed Architectural Designs

A New Java, Spring Boot Service within HealthSource Repository

Access to backend actions and data via backend HTTP APIs

Improving / overhauling backend services as needed

Incorporating the IntakeService endpoints so as to present a singular, external API

“Smoothing” out the difficulties of interacting with the various APIs

Emitting Auditing Events directly to the queue

Known Unknowns

Authentication and Authorization-Scoping Mechanisms

Helm / Azure configurations for “external” access from the Athena BFF

What are the implications of almost-duplicate audit tracking events?

Can we deploy this system without an active connection to the database?

How will we replace the need for TrayApp for document upload?

What are we going to call it?

Size Estimate

Confidence

Risks/Mitigations

Rollout Mechanics

Security Considerations and Implications

Monitoring, Alerting, and Measuring

Testing

Dependencies

Purpose of this Document

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we’ll balance many competing priorities to achieve a solution that delivers optimal business value.

Relevant links

- Requirements *summary*: 📄 [Requirements for ROI team in Athena Phase 2](#)
- Athena Integration Tech Spec: 📄 [Tech Spec - Embedded ROI EHR Integration \(Athena\)](#)
- UI Designs: [Figma](#)

- [PRD](#)
- Overarching Project LucidChart: [LucidChart Workflow](#)

Section Owner: @Jonathan Falkenstein and @Alagu Manikandan Esakkimuthu

✓ What is this?

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

System Context

✓ What is this section?

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.

What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

This Tech Spec addresses the requirements of reflected at [👉 Requirements for ROI team in Athena Phase 2](#).

The current “state of the world” is that there are a lot of different services and API endpoints within the “HealthSource” domain and many of them are not suitable for external consumption, for several reasons:

- Some don’t support direct service-to-service authentication and instead be focused on direct consumption by browser users.
- Some required complicated logic or conditional considerations, making it challenging to interact with
- They often don’t provide the exact features we require for the requirements of the Athena project.
- They utilize legacy systems and patterns that lend themselves maintenance and scalability concerns.

With that said, the majority of the required functionality *does* already exist within the various HealthSource systems. Thus, re-implementing all the functionality required would be wasteful.

As a result, **there is a need for a coherent, externally available API interface for all of HealthSource.** This API interface will provide for the needs of external consumers such as the

Athena SMART App (and other external users in the future).

The timeline provided for this work is very tight (4-6 weeks).

Design Goals

▼ What is this section?

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.

What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

The API layer we will create will sit at the external boundary of HealthSource systems, providing a **consolidated interface for external consumers**.

There is a short-term, immediate focus goal for this service as well as longer term vision for its role.

In the short-term...

This will be a **thin orchestration API layer** that will mostly proxy through to the various back-end services that already exist. *Hopefully, this service won't need direct access to the database at all.* This might not be able to be accomplished.

The point of this layer is so that we can **get a deliverable for the business quickly**, but making available production-ready API endpoints to be consumed by the Athena project.

In the medium-to-long-term...

Important: The immediate goal is creating this thin orchestration layer. However, it is expected that after initial delivery, we'll evaluate performance and functionality of the service and identify backend services being invoked that should be fixed / overhauled / migrated / replaced.

This external orchestration layer will allow us to do those fixes, migrations, and replacements of the various backend services without affecting the external API contracts we have via the orchestration layer. In other words, **the API orchestration layer will give us room to perform the important, deeper changes in the backend on our own time schedule** without the tight time constraints of this project.

Proposed Architectural Designs

▼ What is this section?

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.

What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

A New Java, Spring Boot Service within HealthSource Repository

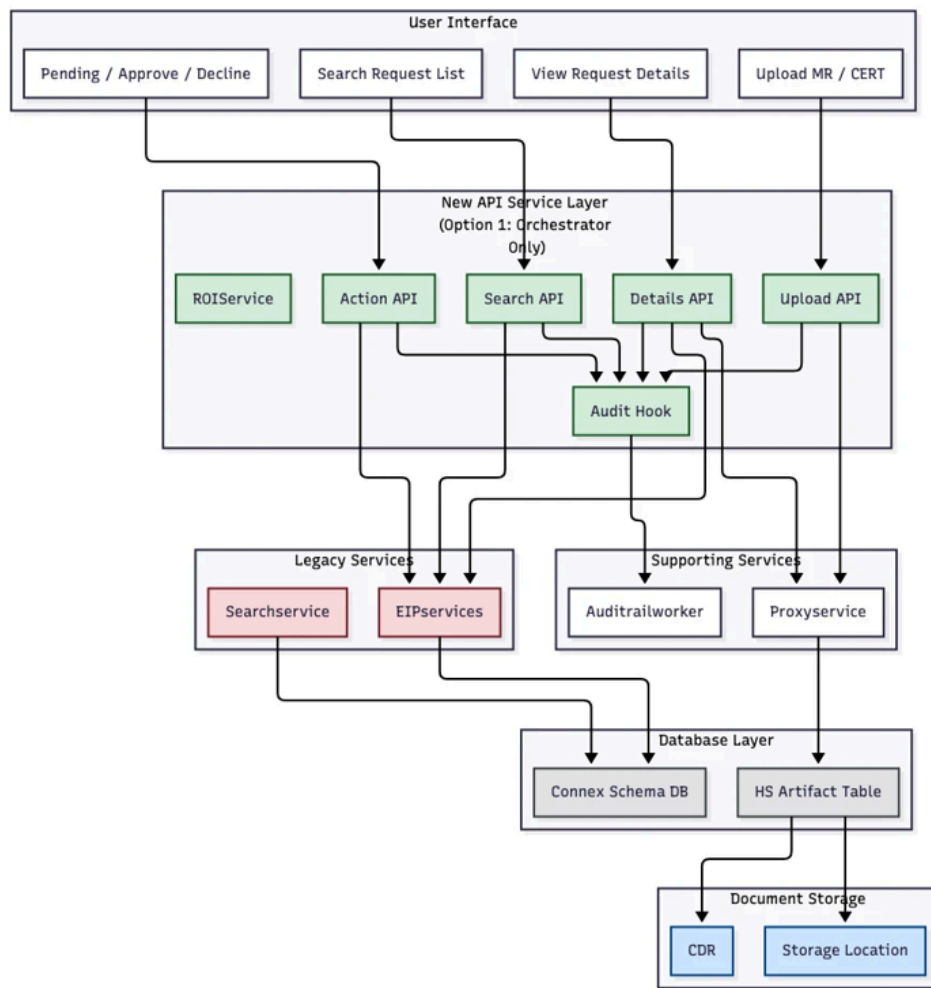
As described above, the goal is to create a **thin orchestration API for external consumers of the HealthSource services**. Thus, it will make sense to co-locate it within HealthSource and leverage HealthSource's existing deployment, feature-flagging, and observability tooling. This will accelerate development for the team, which is important for our time constraints.

Despite being within the HealthSource ecosystem, **this service will expressly avoid entanglement with other HealthSource code via Java dependencies**. Instead, most required functionality will be delegated to the other existing API systems within HealthSource.

Ideally, we would utilize a more modern version of Java, though it's unclear if this would work if we're co-located within HealthSource.

Access to backend actions and data via backend HTTP APIs

This orchestration API will accomplish all required actions and access data via existing APIs on services like the SearchService, EIPServices, ProxyService, and others. The *goal and hope* is that it will not need to directly access the database. This will enable easier / faster / more stable scaling to handle changes in usage. It will also make it so that **we can be “production-ready” faster**, since we’d be using already-battle-tested APIs.



Improving / overhauling backend services as needed

As already documented in [HEAL-1346 Athena request list](#), there are feature gaps in some of the underlying backend services, like SearchService that (as they exist) will not meet project requirements. In these cases, we’ll need to update those underlying services to add the features we require. The hope is these updates will be minimal and non-breaking changes.

Incorporating the IntakeService endpoints so as to present a singular, external API

Athena Phase 1 included the creation of the IntakeService. This is an important backend service. While it might have initially been created as being externally accessible, it would be more consistent and provide a better external interface if this new Orchestrator had endpoints to

cover the scope of IntakeServices, even if those endpoints are simply proxying to the backend service.

“Smoothing” out the difficulties of interacting with the various APIs

One of the greatest benefits this orchestrator API can provide is handling the essential conditional logic and sequencing of which requests to make, from which services, with which parameters, and when.

For example, as documented in [HEAL-1347 Athena request detail API and document access patterns](#), the process and sources for obtaining artifacts is different depending on the particular eRequest status. In at least one case, it requires making a separate request for every page of artifact to be retrieved.

A core part of the “mission” for this orchestrator API is to provide singular, coherent, and logical API contracts for external “HealthSource” consumers.

Emitting Auditing Events directly to the queue

Existing research spike: [HEAL-1349 Athena Audit Trail Implementation Research](#)

Audit tracking events will be emitted by the orchestrator directly into the Azure Storage Queue that is read by the AuditTrailWorker (which actually records those events to the database).

Most (but not all) of the auditable actions from the requirements do have corollaries in HealthSource. However, these do not include the required datapoints in them, such as the source EHR and Athena User ID. Therefore, **we will create a series of External* audit events** to track these actions:

- ExternalRequestViewed
- ExternalCertificationUploaded
- ExternalMedicalRecordsUploaded
- ExternalPracticeReviewApproved
- ExternalPracticeReviewDeclined
- ExternalCertificationApproved
- ExternalCertificationDeclined
- ExternalNoRecordsFound

For each of these audit events, we'll pass the Athena User Name as the `userName` and the Athena user ID as the `user id` with `"source": "athena"` in the `eventData`. These are all attributes required by the Athena PRD but not available in the existing events.

Known Unknowns

▼ What is this section?

This section documents the current gaps in our knowledge that we need to research in order to properly develop this.

These are some larger, notable things that are not currently known/documented for this project. There are definitely other things not known, but these are important ones that have architectural and timeline implications. Some of these things might be already known (in part or in full) by certain individuals, but the plan is to gather this information into research documents that will need to inform our choices going forward.

Authentication and Authorization-Scoping Mechanisms

Research task: [🔗 HEAL-1509: Discovery: How will/should authentication and authorization-scoping work](#) TO DO

There are two major “legs” communication when external services are interacting with the Orchestration API:

- Between the external service and the orchestration API
- Between the orchestration API and the other backend services it will be invoking

For each of these legs, we need to determine answers to questions like:

- How do we determine “who” / “what” is making the request?
- How do we scope the particular data returned or affected to the right entity/entities?

Most importantly, if we're using service account authentication, how can we do this in a way that retains the real “who” behind the originating request during that second leg. In cases where existing endpoints from backing services are permission-scoped based upon the requester, we will need to research the mechanism of that user attribution from the request context and...

- *(If there's a way to attribute the originating user on a service-to-service call)* be sure this avenue is used for all relevant backend requests
- *(If there's NOT already a way to attribute this),* we'll want to create new, near-duplicate endpoints with identical functionality but that derive the current user from the request payload. **We shouldn't plan to modify the existing permissions framework due to the headwinds we'd face from security on that.**

Research Task: 🔗 [HEAL-1514: Discovery: How can we make the External API accessible to the Snuffy AWS-hosted BFF service?](#) WON'T DO

We will need to determine how to make this orchestrator API accessible to the Athena Backend-For-Frontend (BFF) server, which will be hosted in our Snuffleupagus-Prod AWS account. There will need to be certain Helm (and maybe even Azure) configurations to support this. It might also require some assistance from the Platform team and / or AppSec review in order to make this possible. **We should identify if there will be dependencies on other teams in order to deliver our application.**

This shouldn't be a *difficult* investigation, but we will need to determine the specifics of what will be necessary to ensure this service is accessible to those it needs to be accessible to without being *overly* accessible.

What are the implications of almost-duplicate audit tracking events?

Research Task: 🔗 [HEAL-1515: Discovery: What are the implications of near-duplicate audit events?](#) WON'T DO

If the Orchestrator API is emitting its *own* “External*” audit events and the underlying services it's invoking are (sometimes) emitting other events for similar situations, what is the effect on user experience? What is the effect on other systems? Is this acceptable?

These new events are “near-duplicate” but not duplicate because they will be associated with source EHR (Athena) details such as Athena user display name and id. This is in contrast with the existing events which will (likely) be associated with the service account for the API rather than the originating user.

Can we deploy this system without an active connection to the database?

Research Task: 🔗 [HEAL-1516: Discovery: Would a thin orchestrator API be possible without a direct connection to the database?](#) WON'T DO

The initial vision for this system is one that does not have direct access to a database and instead can delegate its database needs to the underlying backend services it will be interacting with. However, it is not currently known if this will be possible. For example, it could be that performing proper authorization checks or creating the required payloads for emitting audit trail events will require access to the database. We'll need to take an inventory of our all essential functions of this API and determine whether any of those functions require a database connection.

If it *will* require access to a database, this will bring with it the extra configuration, dependency, and frameworking overhead of interacting with it. It also will mean implications for additional database load and connection limits.

How will we replace the need for TrayApp for document upload?

Research Task: [🔗 HEAL-1520: Discovery and Design: How will we accomplish efficient document upload without the use of TrayApp?](#) WON'T DO

Right now, document uploads are handled via TrayApp, which will chunk and stream sometimes very large files to our artifact storage systems. Athena's SMART App will not have access to TrayApp. Therefore, we need to investigate alternatives. The important consideration here is that these files can be very large. We cannot and should not try any approach that requires having all file bytes in memory at once, as this could quickly destabilize the API. We also shouldn't expect the consuming applications to do this chunking and streaming on their end (if we can at all avoid it).

What are we going to call it?

We've had some suggestions already:

- ROI Service
- HealthSource API

This document keeps calling it an "Orchestrator API" or "External API" but that's a bit clumsy and unclear. Input is welcome here.

Ideally, the name makes clear "This is intended as the external API interface to HealthSource systems for external consumers as a one-stop-shop."

Size Estimate

▼ What is this section?

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.

What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Person-Sprint Estimate

- API Planning / Research: 1
- Service scaffolding / infrastructure / deployment: .5
- API Development: 2
- Updating underlying services with required changes: 2
- Audit Tracking: 1
- Observability Implementation / Dashboarding: 1
- Validation / Final Bugfixes / Validation: 1

Approximate Estimate: 8.5 person-sprints

This means that we'd expect this to take roughly 4 sprints for a two-person team developing this.

Caveat: *This does not account for time / effort to get this through any sort of AppSec review. If this ends up being required, it could prolong time-to-delivery.*

Confidence

▼ What is this section?

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.

What: One or two sentences describing how confident we are about our size estimate.

So long as we can keep the orchestrator API a **thin** layer that mostly delegates its core functionality to the underlying backend services, there shouldn't be significant surprises. The largest source of possible overruns will be in modifying the underlying services. Some of these have significant development friction for numerous reasons as well as risk.

Risks/Mitigations

▼ What is this section?

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.

What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	Mitigation
This work is flagged by AppSec as needing special review / documentation, etc which would delay delivery.	We should loop in @Tyler Beach ASAP once this document is ready for review and do our best to fast-track this through the process (since we'll be utilizing existing security / deployment frameworks).
Supporting Service-Account-With-User authentication / authorization is not supported.	Rather than modifying existing security code, we'll simply create new endpoints that authorize somewhat differently. Research into this will be carried out in HEAL-15 09: Discovery: How will/should authentication and authorization-scoping work TO DO.
Database interaction will be required for essential functions of this API. This could extend development timelines some and add complexities around database access and scaling.	This is likely already a “solved problem” with many existing patterns for this. I suspect, while this adds complexity, it shouldn't be unmanageable.
It's likely we'll need to modify existing backend services to add the features we require that aren't currently supported. This should be minimal, but there will be some. There are often development “headwinds” when working in some of these legacy codebases and there's the added risk that any changes made could affect many other systems.	The mitigation strategy here is to make the smallest and least-intrusive set of changes possible without adding to the mess that's already there. There are patterns that could be followed such as writing a suite of unit tests FIRST before making any changes to the system in order to prevent accidental breakage. Changes we need to make like this will unfortunately be slow-going.
This pathway WILL add load to our existing systems, some of which are	Initial delivery exposes us to this risk, but we expect minimal traffic

already under notable load. This poses a nonzero risk of instability to existing production systems.

after this initial delivery. After initial delivery, we can evaluate performance characteristics and make plans for more significant heavy lifting and changes without altering our new API contracts. Thus, we can iterate on this plan to address any observed sources of production instability.

Rollout Mechanics

✓ What is this section?

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.

What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.

If rollback is for some reason not possible in a step, note that here so we can understand the implications.

Since the goal is for a thin API layer that is simply orchestrating calls to the existing, production-ready, battle-tested backend service APIs, we should be able to put it together quickly with minimal risk to our timeline or existing systems. Once we put up our API, we'll mostly be fully delivered.

We can back-load work on rolling out observability tooling and monitoring for after initial delivery to get something up and usable sooner, but this probably will only shave off a week or two.

Security Considerations and Implications

▼ What is this section?

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.

What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

This orchestration API will need to authenticate and authorize requests from our Snuffleupagus-Prod AWS account where the BFF server will be making requests.

- AuthN and AuthZ concerns are TBD, subject to the [linked research task](#).
- It's unclear at the moment how/if Athena users will be mapped to HealthSource users or otherwise how we'll scope permissions to the Athena users.
- We'll need to support service account authentication but somehow scoped to specific Athena/HealthSource users.

Monitoring, Alerting, and Measuring

▼ Click here to expand...

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.

What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

- We'll want a "standard suite" of API metrics (request counts, latency, request sizes, statuses, errors)
- We'll want to institute DataDog APM on the service.
 - *Ideally (if feasible)* we'll want to support *distributed tracing* in a way where the API requests can have a parent span from the BFF and can have as child spans the tracing from downstream APIs that our API will call. It's unclear at the moment if this is supported within

HealthSource or the BFF right now, especially when the BFF's datadog account might be different (i.e. Barkley).

- We'll definitely want HealthSource-standard logging practices on all requests.
- We'll want to research and create relevant DataDog monitors for tracking things like persistent errors and other conditions.
 - This will require exploring existing patterns in HealthSource.

Testing

✓ What is this section?

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.

What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Since this will be mostly reliant upon greenfield development on API code that makes requests to the underlying backend services, **all new code will include unit tests** that corresponds with it.

Furthermore, the **base urls of each of the backend services we intend to call will be configurable** via environment variables so that those requests can be redirected and intercepted for use with testing fixtures.

There are existing End-to-End testing systems / patterns in place in the codebase that could be explored. However, for the time being and considering the tight timeline for this project, I don't think it's worthwhile as something to invest significant time into.

Dependencies

✓ Click here to expand...

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.

What: A bullet point list of external dependencies that we will need in order for this project to be successful.

Because the plan is to largely make available some API endpoints that use existing HealthSource endpoints and frameworking that already exists in production, there should be

minimal (if any) notable inter-team dependencies.

The only possible exception to this is if we need Platform and/or AppSec coordination to deploy this API in a way accessible outside of our Azure cloud.